

SAPIENZA
UNIVERSITÀ DI ROMA

Combinatorial Optimization

Catalogue of combinatorial problems

All the problems defined in the course, grouped by family: for each one its definition and its assumptions, as in the chapters of the lecture notes.

Fabio Furini

fabiofurini.github.io

Department of Computer, Control and Management Engineering — Sapienza University of Rome

Online version and downloadable material:
fabiofurini.github.io/combinatorial-optimization/
github.com/fabiofurini/combinatorial-optimization

Academic year 2026–2027

Contents

1 Central problems **3**

1.1 The knapsack problem (KP) . . . 3
1.2 The bin packing problem (BPP) . . . 4
1.3 The vertex coloring problem (VCP) . . . 4
1.4 The maximum clique problem (MCP) . . . 5
1.5 The travelling salesman problem (TSP) . . . 6
1.6 Scheduling on identical parallel machines ($P\|C_{\max}$) . . . 7
1.7 Capacitated lot sizing (CLS) . . . 7

2 Knapsack family **9**

2.1 Subset sum problem (SSP) . . . 9
2.2 Bounded knapsack problem (BKP) . . . 10
2.3 Unbounded knapsack problem (UKP) . . . 11
2.4 Multidimensional knapsack problem (MDKP) . . . 11
2.5 Multiple knapsack problem (MKP) . . . 12
2.6 Multiple-choice knapsack problem (MCKP) . . . 13
2.7 Knapsack problem with setups (KPS) . . . 13
2.8 Rod cutting problem (RCP) . . . 14
2.9 Coin change problem (CCP) . . . 15

3 Packing **16**

3.1 Cutting stock problem (CSP) . . . 16

4 Graphs **17**

4.1 Maximum stable set problem (MSSP) . . . 17
4.2 Vertex-weighted maximum clique problem (VWMCP) . . . 18
4.3 Edge-weighted maximum clique problem (EWMCP) . . . 18

5 Routing **19**

5.1 Metric travelling salesman problem (metric TSP) . . . 19
5.2 Asymmetric travelling salesman problem (ATSP) . . . 19
5.3 Hamiltonian cycle problem (HCP) . . . 19

6 Scheduling **20**
6.1 Scheduling on a fixed number of identical machines ($Pm\|C_{\max}$) 20
6.2 Scheduling on parallel machines with different speeds ($Q\|C_{\max}$) 20
6.3 Scheduling on unrelated parallel machines ($R\|C_{\max}$) 20

7 Lot sizing **21**
7.1 Uncapacitated lot sizing problem (ULS) 21
7.2 Capacitated lot sizing with constant capacity (CLS-C) 21
7.3 Lot sizing with several products and setup times (MLS) 22
7.4 Lot sizing with initial stock (LS-SI) 22
7.5 Lot sizing with overtime (LS-OT) 22
7.6 Lot sizing with unsatisfied demand (LS-UD) 22

Chapter 1

Central problems

1.1 The knapsack problem (KP)

Definition 1.1: The knapsack problem

Given:

1. a knapsack of *capacity* $c \in \mathbb{Z}_{\geq 1}$,
2. a set $\mathscr{I} = \{1, 2, \ldots, n\}$ of n items,
3. for each item $j \in \mathscr{I}$, a *profit* $p_j \in \mathbb{Z}_{\geq 1}$ and a *weight* $w_j \in \mathbb{Z}_{\geq 1}$,

the **Knapsack Problem** (KP) asks to decide which items to insert into the knapsack, so that the total weight of the selected items does not exceed the knapsack capacity, and the total profit of the selected items is maximized.

Assumptions

- We make the following assumptions:
  1. The total weight of all items exceeds the knapsack capacity (otherwise, all items could be selected). Formally:
  $$\sum_{j \in \mathscr{I}} w_j > c$$
  2. Each item individually fits into the knapsack (otherwise, it cannot be selected). Formally:
  $$w_j \leq c, \quad \forall j \in \mathscr{I}$$
- Defined in Chapter 2 «Knapsack problems».
- The course on this problem →.

1.2 The bin packing problem (BPP)

Definition 1.2: The bin packing problem

Given:

1. an infinite number of bins, each with *capacity* $c \in \mathbb{Z}_{\geq 1}$,
2. a set $\mathscr{I} = \{1, 2, \ldots, n\}$ of n items,
3. for each item $j \in \mathscr{I}$, a *weight* $w_j \in \mathbb{Z}_{\geq 1}$,

the **Bin Packing Problem** (BPP) asks to decide in which bin to place each item, so that the total weight of the items in any bin does not exceed the bin capacity, and the total number of bins used is minimized.

Assumptions

- We make the following assumptions:
  1. The total weight of all items exceeds the bin capacity (otherwise, all items could fit in a single bin). Formally:
  $$\sum_{j \in \mathscr{I}} w_j > c$$
  2. Each item individually fits into a single bin (otherwise, it cannot be packed at all). Formally:
  $$w_j \leq c, \qquad \forall j \in \mathscr{I}$$
- Defined in Chapter 3 «Bin packing and cutting stock».
- The course on this problem →.

1.3 The vertex coloring problem (VCP)

Definition 1.3: The vertex coloring problem

Given:

1. a simple *graph* $G = (V, E)$, where
  $$V = \{1, 2, \ldots, n\}$$
  is the set of n *vertices*, and
  $$E \subseteq \big\{ \{u, v\} : u, v \in V,\ u \neq v \big\}$$
  is the set of m *edges*,

the **Vertex Coloring Problem** (VCP) asks to decide how to assign a color to each vertex, so that no two adjacent vertices share the same color, and the total number of colors used is minimized.

Assumptions

- We make the following assumptions:

  1. The graph has no isolated vertices. An isolated vertex can receive any color used by the rest of the graph; if the graph has no edges, the optimal value is one. Formally:

  $$\deg(v) \geq 1, \qquad \forall\, v \in V.$$

  2. No vertex has its neighbourhood contained in the neighbourhood of another vertex. Formally:

  $$N(u) \not\subseteq N(v), \qquad \forall\, u, v \in V,\ u \neq v.$$

- The second assumption can be made without loss of generality, because:

  1. if $N(u) \subseteq N(v)$, the vertex u can be removed without changing the chromatic number: every coloring of $G \setminus \{u\}$ extends to u with the color of v;
  2. the vertices u and v are not adjacent: otherwise $v \in N(u) \subseteq N(v)$, which is impossible in a simple graph, so u and v can share a color.

- Defined in Chapter 4 «Vertex coloring».
- The course on this problem →.

1.4 The maximum clique problem (MCP)

Definition 1.4: The maximum clique problem

Given:

1. a simple *graph* $G = (V, E)$, where

$$V = \{1, 2, \ldots, n\}$$

is the set of n *vertices*, and

$$E \subseteq \big\{\, \{u, v\} : u, v \in V,\ u \neq v \,\big\}$$

is the set of m *edges*,

the **Maximum Clique Problem** (MCP) asks to find a clique $K \subseteq V$ of G whose number of vertices is maximum. Its optimal value is the clique number $\omega(G)$.

Assumptions

- We consider nontrivial instances: the graph has at least one edge and at least one non-edge. Formally:

$$1 \leq m < \binom{n}{2}.$$

- The assumption can be made without loss of generality: if the graph has no edge, every single vertex is a maximum clique and $\omega(G) = 1$; if the graph is complete, V itself is a clique and $\omega(G) = n$.

- Defined in Chapter 5 «The maximum clique problem».

- The course on this problem →.

1.5 The travelling salesman problem (TSP)

Definition 1.5: The travelling salesman problem

Given:

1. a set $\mathscr{V} = \{1, 2, \ldots, n\}$ of n *cities*,
2. for each pair of distinct cities $i, j \in \mathscr{V}$, a *distance* $d_{ij} \in \mathbb{Z}_{\geq 1}$, with $d_{ij} = d_{ji}$,

the **travelling salesman problem** (TSP) asks to find a closed route that starts from a city, visits every other city exactly once and returns to the starting city, so that the total distance travelled is minimized.

Assumptions

- We make the following assumptions:

1. There are at least three cities (with one or two cities there is only one route). Formally:
$$n \geq 3$$
2. The distances are symmetric (travelling from i to j costs as much as travelling from j to i). Formally:
$$d_{ij} = d_{ji}, \qquad \forall i, j \in \mathscr{V},\ i \neq j$$

- Defined in Chapter 6 «The travelling salesman problem».

- The course on this problem →.

1.6 Scheduling on identical parallel machines ($P||C_{\max}$)

Definition 1.6: Scheduling on identical parallel machines

Given:

1. a set of m identical *machines*,
2. a set $\mathscr{J} = \{1, 2, \ldots, n\}$ of n *jobs*,
3. for each job $j \in \mathscr{J}$, a *processing time* $t_j \in \mathbb{Z}_{\geq 1}$,

the problem $P\,||\,C_{\max}$ asks to assign each job to a machine, so that the time needed to complete all the jobs, i.e., the largest total processing time assigned to a machine, is minimized.

Assumptions

- We make the following assumption:
  1. There are more jobs than machines (otherwise each job gets a machine of its own, and the optimal makespan is $\max_j t_j$). Formally:

$$n > m$$

- Defined in Chapter 7 «Parallel machine scheduling».
- The course on this problem →.

1.7 Capacitated lot sizing (CLS)

Definition 1.7: Capacitated lot sizing

Given:

1. a set $T = \{1, 2, \ldots, m\}$ of m *time periods*,
2. for each period $t \in T$, a *demand* $d_t \in \mathbb{Z}_{\geq 1}$ and a *capacity* $cap_t \in \mathbb{Z}_{\geq 1}$, both counted in units of one product,
3. for each period $t \in T$, three non-negative integer costs: the *setup cost* sc_t, the *variable production cost* vc_t per unit produced and the *holding cost* hc_t per unit kept in stock at the end of the period,

the **capacitated lot sizing problem** (CLS) asks to decide how much to produce in every period, so that every demand is satisfied with what is produced and what is in stock, the capacities are respected, and the total cost of setups, production and holding is minimized.

Assumptions

- We make the following assumptions:

1. The stock is empty at the beginning and at the end of the horizon. Leaving stock at the end only adds cost, so no optimal plan needs it; an initial stock is a variant of the problem (next section).
2. The capacities allow to satisfy the demands, period after period (otherwise there is no feasible plan). Formally:

$$\sum_{u=1}^{t} cap_u \geq \sum_{u=1}^{t} d_u, \qquad \forall t \in T$$

3. The quantities are integer. With integer data, once the periods with a setup are fixed, what remains is a minimum cost flow problem, which has an integer optimal solution: restricting the quantities to integers does not change the optimal cost.

- Defined in Chapter 8 «Capacitated lot sizing».
- The course on this problem →.

Chapter 2

Knapsack family

2.1 Subset sum problem (SSP)

Definition 2.1: Subset sum problem

Given:

1. a target value $c \in \mathbb{Z}_{\geq 1}$,
2. a set $\mathscr{I} = \{1, 2, \ldots, n\}$ of n items,
3. for each item $j \in \mathscr{I}$, a *weight* $w_j \in \mathbb{Z}_{\geq 1}$,

the **Subset Sum Problem** (SSP) asks whether there exists a subset of items whose total weight is exactly equal to the target value c.

Assumptions

- We make the following assumptions:
  1. The total weight of all items is at least the target value (otherwise, no feasible subset can exist). Formally:
  $$\sum_{j \in \mathscr{I}} w_j \geq c$$
  2. Each item weight is not larger than the target value (otherwise, it cannot belong to any feasible subset). Formally:
  $$w_j \leq c, \quad \forall j \in \mathscr{I}$$
- Defined in Chapter 2 «Knapsack problems».

2.2 Bounded knapsack problem (BKP)

Definition 2.2: Bounded knapsack problem

Given:

1. a knapsack of *capacity* $c \in \mathbb{Z}_{\geq 1}$,
2. a set $\mathscr{I} = \{1, 2, \ldots, n\}$ of n items,
3. for each item $j \in \mathscr{I}$, a *profit* $p_j \in \mathbb{Z}_{\geq 1}$, a *weight* $w_j \in \mathbb{Z}_{\geq 1}$, and a *number of available copies* $b_j \in \mathbb{Z}_{\geq 1}$,

the **Bounded Knapsack Problem** (BKP) asks to decide how many copies of each item to insert into the knapsack, so that the total weight of all selected item copies does not exceed the knapsack capacity, each item is taken no more than its available copies, and the total profit of all selected item copies is maximized.

Assumptions

- We make the following assumptions:

  1. The total weight of all available copies of all items exceeds the knapsack capacity (otherwise, all copies could be selected). Formally:
  $$\sum_{j \in \mathscr{I}} b_j \, w_j > c$$
  2. Each item individually fits into the knapsack (otherwise, it cannot be selected). Formally:
  $$w_j \leq c, \qquad \forall j \in \mathscr{I}$$
  3. The total weight of all available copies of any single item does not exceed the knapsack capacity (otherwise, that item could never be taken in the maximum allowed number of copies). Formally:
  $$b_j \, w_j \leq c, \qquad \forall j \in \mathscr{I}$$

- Defined in Chapter 2 «Knapsack problems».

2.3 Unbounded knapsack problem (UKP)

Definition 2.3: Unbounded knapsack problem

Given:

1. a knapsack of *capacity* $c \in \mathbb{Z}_{\geq 1}$,
2. a set $\mathscr{I} = \{1, 2, \ldots, n\}$ of n items,
3. for each item $j \in \mathscr{I}$, which is available in an unlimited number of copies, a *profit* $p_j \in \mathbb{Z}_{\geq 1}$ and a *weight* $w_j \in \mathbb{Z}_{\geq 1}$,

the **Unbounded Knapsack Problem** (UKP) asks to decide how many copies of each item to insert into the knapsack, so that the total weight of all selected item copies does not exceed the knapsack capacity and the total profit of all selected item copies is maximized.

Assumptions

- We make the following assumptions:
  1. The total weight of all possible copies of all items (up to the maximum number that can fit in the knapsack) exceeds the knapsack capacity (otherwise, all copies could be selected). Formally:
  $$\sum_{j \in \mathscr{I}} \hat{b}_j \, w_j > c$$
  2. Each item individually fits into the knapsack (otherwise, it cannot be selected). Formally:
  $$w_j \leq c, \qquad \forall j \in \mathscr{I}$$
- Defined in Chapter 2 «Knapsack problems».

2.4 Multidimensional knapsack problem (MDKP)

Definition 2.4: Multidimensional knapsack problem

Given:

1. a k-dimensional knapsack,
2. for each dimension $d \in \underbrace{\{1, 2, \ldots, k\}}_{=\mathscr{D}}$, a *capacity* $c_d \in \mathbb{Z}_{\geq 1}$,
3. a set $\mathscr{I} = \{1, 2, \ldots, n\}$ of n items,
4. for each item $j \in \mathscr{I}$, a *profit* $p_j \in \mathbb{Z}_{\geq 1}$,
5. for each dimension $d \in \mathscr{D}$ and each item $j \in \mathscr{I}$, a *weight* $w_{dj} \in \mathbb{Z}_{\geq 1}$,

the **Multidimensional Knapsack Problem** (MDKP) asks to decide which items to insert into the knapsack, so that for every dimension the total weight of the selected items

does not exceed the corresponding capacity, and the total profit of the selected items is maximized.

Assumptions

- We make the following assumptions:

  1. The set of all items does *not* fit in the knapsack (otherwise, selecting all items is feasible and optimal). Formally,
  $$\exists\, d \in \mathscr{D} \text{ such that } \sum_{j \in \mathscr{I}} w_{dj} \;>\; c_d.$$
  2. Each item individually fits in the knapsack in every dimension (otherwise, it can never be selected). Formally,
  $$w_{dj} \leq c_d, \qquad \forall\, d \in \mathscr{D},\ \forall\, j \in \mathscr{I}.$$

- Defined in Chapter 2 «Knapsack problems».

2.5 Multiple knapsack problem (MKP)

Definition 2.5: Multiple knapsack problem

Given:

1. $m \in \mathbb{Z}_{\geq 1}$ knapsacks,
2. for each knapsack $k \in \underbrace{\{1, 2, \ldots, m\}}_{=\mathscr{K}}$, a *capacity* $c_k \in \mathbb{Z}_{\geq 1}$,
3. a set $\mathscr{I} = \{1, 2, \ldots, n\}$ of n items,
4. for each item $j \in \mathscr{I}$, a *profit* $p_j \in \mathbb{Z}_{\geq 1}$ and a *weight* $w_j \in \mathbb{Z}_{\geq 1}$,

the **Multiple Knapsack Problem** (MKP) asks to decide which items to insert into each knapsack, so that in every knapsack the total weight of the assigned items does not exceed its capacity, and the total profit of the selected items is maximized.

Assumptions

- We make the following assumptions:

  1. The set of all items does *not* fit in the knapsacks altogether (otherwise, selecting all items is feasible and optimal). Formally,
  $$\sum_{j \in \mathscr{I}} w_j \;>\; \sum_{k \in \mathscr{K}} c_k.$$
  2. Each item individually fits in (at least) one knapsack (otherwise, it can never be selected). Formally,
  $$\exists\, k \in \mathscr{K} \text{ such that } w_j \leq c_k, \qquad \forall\, j \in \mathscr{I}.$$

- Defined in Chapter 2 «Knapsack problems».

2.6 Multiple-choice knapsack problem (MCKP)

Definition 2.6: Multiple-choice knapsack problem

Given:

1. a knapsack of *capacity* $c \in \mathbb{Z}_{\geq 1}$,
2. a set $\mathscr{I} = \{1, 2, \ldots, n\}$ of n items,
3. the set of items is partitioned into $q \in \mathbb{Z}_{\geq 2}$ classes,
4. for each class $h \in \{1, 2, \ldots, q\}$, the subset $\mathscr{J}_h \subseteq \mathscr{I}$ contains the items of that class,
5. for each item $j \in \mathscr{I}$, a *profit* $p_j \in \mathbb{Z}_{\geq 1}$ and a *weight* $w_j \in \mathbb{Z}_{\geq 1}$,

the **Multiple-Choice Knapsack Problem** (MCKP) asks to decide which items to insert into the knapsack, so that exactly one item is selected from each class, the total weight of the selected items does not exceed the knapsack capacity, and the total profit of the selected items is maximized.

Assumptions

- We make the following assumptions:
  1. Each class contains at least one item (otherwise, selecting exactly one item from each class is impossible). Formally:
  $$\mathscr{J}_h \neq \emptyset \qquad \forall\, h \in \{1, \ldots, q\}.$$
  2. The instance is feasible, i.e., there exists at least one choice of exactly one item per class that fits in the knapsack. A necessary and sufficient (and easy-to-check) condition is:
  $$\sum_{h=1}^{q} \min_{j \in \mathscr{J}_h} w_j \;\leq\; c.$$
- Defined in Chapter 2 «Knapsack problems».

2.7 Knapsack problem with setups (KPS)

Definition 2.7: Knapsack problem with setups

Given:

1. a knapsack of *capacity* $c \in \mathbb{Z}_{\geq 1}$,
2. a set $\mathscr{I} = \{1, 2, \ldots, n\}$ of n items,
3. the set of items is partitioned into $q \in \mathbb{Z}_{\geq 2}$ classes,
4. for each class $h \in \{1, 2, \ldots, q\}$, the subset $\mathscr{J}_h \subseteq \mathscr{I}$ contains the items of that class, with an associated *setup cost* $f_h \in \mathbb{Z}_{\geq 1}$ and a *setup weight* $s_h \in \mathbb{Z}_{\geq 1}$,

5. for each item $j \in \mathscr{I}$, a *revenue* $p_j \in \mathbb{Z}_{\geq 1}$ and a *weight* $w_j \in \mathbb{Z}_{\geq 1}$,

the **Knapsack Problem with Setups** (KPS) asks to decide which items to insert into the knapsack, so that the total weight of the selected items plus the setup weights of their classes does not exceed the knapsack capacity, and the total revenue of the selected items minus the setup costs of their classes is maximized.

Assumptions

- We make the following assumptions:

  1. Each class contains at least one item. Formally:

  $$\mathscr{J}_h \neq \emptyset \qquad \forall\, h \in \{1, 2, \ldots, q\}.$$

  2. Each item can fit into the knapsack together with the setup weight of its class (otherwise, it can never be selected). Formally:

  $$w_j + s_h \leq c \qquad \forall\, h \in \{1, 2, \ldots, q\},\ \forall\, j \in \mathscr{J}_h.$$

- Defined in Chapter 2 «Knapsack problems».

2.8 Rod cutting problem (RCP)

Definition 2.8: Rod cutting problem

Given:

1. a rod of *length* $n \in \mathbb{Z}_{\geq 1}$,
2. a set $\{1, 2, \ldots, n\}$ of n possible piece lengths,
3. for each piece length $\ell \in \{1, 2, \ldots, n\}$, a *profit* $p_\ell \in \mathbb{Z}_{\geq 1}$,

the **Rod-Cutting Problem** (RCP) asks to decide how many copies of each piece length to cut, so that the total length of the obtained pieces is equal to the rod length, and the total profit of the obtained pieces is maximized.

- Defined in Chapter 20 «DP for the rod cutting problem».

2.9 Coin change problem (CCP)

Definition 2.9: Coin change problem

Given:

1. a vector $w \in \mathbb{Z}^n_{\geq 1}$ of n *denominations*, each available in unlimited supply,
2. an integer *amount of change* $c \in \mathbb{Z}_{\geq 0}$ to return,

the **Coin Change Problem** (CCP) asks to decide how many coins of each denomination to use so that their total value is exactly equal to c, and the total number of coins used is minimized. If no combination of coins returns exactly c, the instance is *infeasible*.

- Defined in Chapter 21 «DP for the coin change problem».

Chapter 3

Packing

3.1 Cutting stock problem (CSP)

Definition 3.1: Cutting stock problem

Given:

1. an infinite number of bins, each with *capacity* $c \in \mathbb{Z}_{\geq 1}$,
2. a set $\mathscr{I} = \{1, 2, \ldots, n\}$ of n items,
3. for each item $j \in \mathscr{I}$, a *weight* $w_j \in \mathbb{Z}_{\geq 1}$ and a *number of required copies* $d_j \in \mathbb{Z}_{\geq 1}$,

the **Cutting Stock Problem** (CSP) asks to decide in which bin to place each copy of every item, so that the total weight of all item copies in any bin does not exceed the bin capacity, the required number of copies of each item is placed into the bins, and the total number of bins used is minimized.

Assumptions

- We make the following assumptions:
  1. The total weight of all required item copies exceeds the bin capacity (otherwise, all copies could fit in a single bin). Formally:
  $$\sum_{j \in \mathscr{I}} d_j \, w_j \; > \; c.$$
  2. Each item individually fits into a single bin (otherwise, no copy of that item can be packed at all). Formally:
  $$w_j \leq c, \qquad \forall \, j \in \mathscr{I}.$$
- Defined in Chapter 3 «Bin packing and cutting stock».

Chapter 4

Graphs

4.1 Maximum stable set problem (MSSP)

Definition 4.1: Maximum stable set problem

Let $G = (V, E)$ be a graph.

1. A **stable set** (also called **independent set**) is a subset $Q \subseteq V$ of pairwise non-adjacent vertices: $\{u, v\} \notin E$ for all $u, v \in Q$ with $u \neq v$.
2. The **Maximum Stable Set Problem** (MSSP) asks to find a stable set of G whose number of vertices is maximum. Its optimal value is the **stability number** $\alpha(G)$.

Assumptions

- We consider nontrivial instances: the graph has at least one edge and at least one non-edge. Formally:
$$1 \leq m < \binom{n}{2}.$$
- The assumption can be made without loss of generality: if the graph has no edge, every single vertex is a maximum clique and $\omega(G) = 1$; if the graph is complete, V itself is a clique and $\omega(G) = n$.
- Defined in Chapter 5 «The maximum clique problem».

4.2 Vertex-weighted maximum clique problem (VWMCP)

Definition 4.2: Vertex-weighted maximum clique problem

1. Given a graph G and a vertex-weight vector $\boldsymbol{w}$, the **vertex-weighted maximum clique problem** asks to find a clique K of G with maximum total vertex weight $w(K)$.
2. Given a graph G and an edge-weight vector $\boldsymbol{r}$, the **edge-weighted maximum clique problem** asks to find a clique K of G with maximum total edge weight $r(K)$.

- Defined in Chapter 5 «The maximum clique problem».

4.3 Edge-weighted maximum clique problem (EWMCP)

Definition 4.3: Edge-weighted maximum clique problem

1. Given a graph G and a vertex-weight vector $\boldsymbol{w}$, the **vertex-weighted maximum clique problem** asks to find a clique K of G with maximum total vertex weight $w(K)$.
2. Given a graph G and an edge-weight vector $\boldsymbol{r}$, the **edge-weighted maximum clique problem** asks to find a clique K of G with maximum total edge weight $r(K)$.

- Defined in Chapter 5 «The maximum clique problem».

Routing

5.1 Metric travelling salesman problem (metric TSP)

Definition 5.1: Metric travelling salesman problem

A TSP instance is **metric** if its distances satisfy the **triangle inequality**

$$d_{ij} \leq d_{ik} + d_{kj} \qquad \text{for all distinct } i, j, k \in \mathscr{V}.$$

- Defined in Chapter 6 «The travelling salesman problem».

5.2 Asymmetric travelling salesman problem (ATSP)

Definition 5.2: Asymmetric travelling salesman problem

In the **asymmetric travelling salesman problem** (ATSP) the distances are not required to be symmetric: d_{ij} may differ from d_{ji}. The definition of tour and of its length $L(\sigma)$ is unchanged, but a tour and the same tour travelled backwards are different solutions.

- Defined in Chapter 6 «The travelling salesman problem».

5.3 Hamiltonian cycle problem (HCP)

Definition 5.3: Hamiltonian cycle problem

The **Hamiltonian cycle problem** asks whether a graph $G = (V, E)$ has a cycle through all its vertices.

- Defined in Chapter 32 «Approximation for the metric TSP».

Chapter 6

Scheduling

6.1 Scheduling on a fixed number of identical machines ($Pm||C_{\max}$)

Definition 6.1: Scheduling on a fixed number of identical machines

The problem $P\,||\,C_{\max}$ in which the number m of machines is a constant, for example $m = 2$, and only the jobs $\mathscr{J}$ and the processing times $t_j \in \mathbb{Z}_{\geq 1}$ form the input.

- Defined in Chapter 1 «Combinatorial optimization problems».

6.2 Scheduling on parallel machines with different speeds ($Q||C_{\max}$)

Definition 6.2: Scheduling on parallel machines with different speeds

In the three-field notation $\alpha\,|\,\beta\,|\,\gamma$, the machine environment $\alpha = Q$ denotes parallel machines with different speeds; $Q\,||\,C_{\max}$ assigns jobs without additional characteristics to such machines, minimizing the makespan $C_{\max}$.

- Defined in Chapter 7 «Parallel machine scheduling».

6.3 Scheduling on unrelated parallel machines ($R||C_{\max}$)

Definition 6.3: Scheduling on unrelated parallel machines

In the three-field notation $\alpha\,|\,\beta\,|\,\gamma$, the machine environment $\alpha = R$ denotes unrelated parallel machines; $R\,||\,C_{\max}$ assigns jobs without additional characteristics to such machines, minimizing the makespan $C_{\max}$.

- Defined in Chapter 7 «Parallel machine scheduling».

Chapter 7

Lot sizing

7.1 Uncapacitated lot sizing problem (ULS)

Definition 7.1: Uncapacitated lot sizing problem

Given:

1. a set $T = \{1, 2, \dots, m\}$ of $m \in \mathbb{Z}_{\geq 1}$ *time periods*,
2. for each period $t \in T$, a *demand* $d_t \in \mathbb{Z}_{\geq 1}$,
3. for each period $t \in T$, a *setup cost* $sc_t \in \mathbb{Z}_{\geq 0}$, a *variable production cost* $vc_t \in \mathbb{Z}_{\geq 0}$ per unit, and a *holding cost* $hc_t \in \mathbb{Z}_{\geq 0}$ per unit in stock at the end of period t,

the **Uncapacitated Lot-Sizing Problem** (ULS) asks to decide how many units to produce in each period, so that, starting with an empty inventory and ending with an empty inventory, the demand of every period is satisfied, and the total cost of setups, production, and holding is minimized.

- Defined in Chapter 22 «Dynamic programming for lot sizing».

7.2 Capacitated lot sizing with constant capacity (CLS-C)

Definition 7.2: Capacitated lot sizing with constant capacity

The CLS in which the capacity is the same in all periods, $cap_t = C$ for all $t \in T$.

- Defined in Chapter 22 «Dynamic programming for lot sizing».

7.3 Lot sizing with several products and setup times (MLS)

Definition 7.3: Lot sizing with several products and setup times

A set P of products shares the resource of each period: d_{pt}, sc_{pt}, vc_{pt} and hc_{pt} are the data of product $p \in P$ in period $t \in T$; producing a unit of p takes the time $vt_{pt} \in \mathbb{Z}_{\geq 1}$, a setup takes the time $st_{pt} \in \mathbb{Z}_{\geq 0}$, and the capacity cap_t is a time.

- Defined in Chapter 15 «Formulations of capacitated lot sizing».

7.4 Lot sizing with initial stock (LS-SI)

Definition 7.4: Lot sizing with initial stock

The quantity si_p of product p available at the beginning of the horizon is a decision, bought at the cost fc_p per unit; the balance of the first period becomes $si_p + x_{p1} = d_{p1} + s_{p1}$.

- Defined in Chapter 8 «Capacitated lot sizing».

7.5 Lot sizing with overtime (LS-OT)

Definition 7.5: Lot sizing with overtime

The capacity is not fixed: working time in normal hours costs cw_t per unit up to cap_t^N, and overtime costs co_t per unit up to cap_t^O.

- Defined in Chapter 8 «Capacitated lot sizing».

7.6 Lot sizing with unsatisfied demand (LS-UD)

Definition 7.6: Lot sizing with unsatisfied demand

A part of the demand may be left unsatisfied, at a penalty cu_{pt} per unit.

- Defined in Chapter 8 «Capacitated lot sizing».